

Flink 原理详解 - 第 3 讲

Event Time、Processing Time、乱序事件与 Watermark

本讲目标：真正理解“流处理里的时间”为什么比批处理复杂，并能回答：一条迟到 5 分钟的订单到底属于哪个窗口？窗口什么时候能结算？Watermark 到底是什么？

前两讲已经解决了两个问题：为什么需要 Flink，以及一个 DataStream Job 如何从 Source 流向 Sink。本讲开始进入 Flink 最核心的时间语义。

先记一句话：在流处理中，难点不是“现在几点”，而是“业务事件什么时候发生、系统何时无理由相信某段过去已经基本完整”。

同一条订单为什么有两个“时间”？

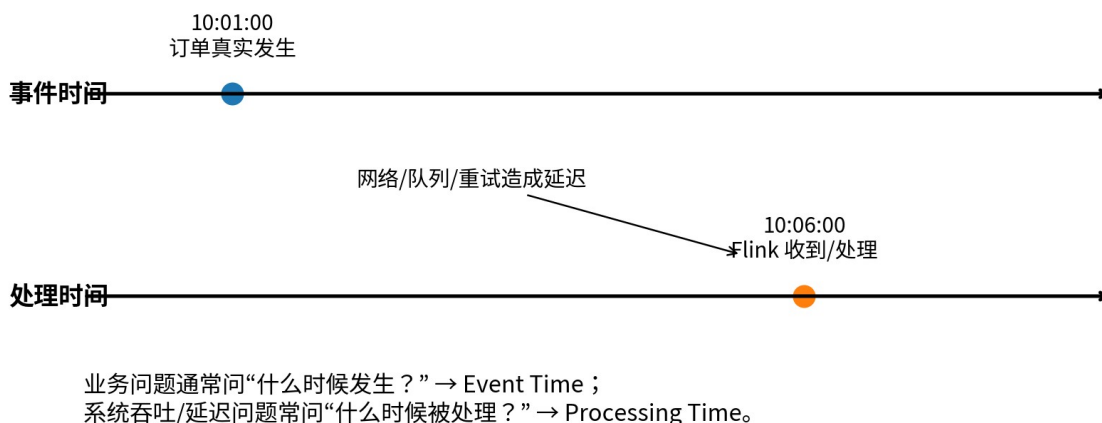


图 1 Event Time 与 Processing Time 是两个不同维度

1. 本讲知识地图

1. 为什么一条事件会同时拥有 Event Time 和 Processing Time。
2. 乱序是怎样产生的，为什么按“到达顺序”统计会产生业务错误。
3. Watermark 不是当前时间，而是事件时间进度信号。
4. 窗口为什么需要 Watermark 才能知道何时触发。
5. 有多个 Kafka 分区/Source Subtask 时，为什么最慢分区会拖住下游 Watermark。
6. 迟到事件与 Watermark 的关系，以及为什么 Watermark 不等于“迟到数据必丢”。
7. 用一个实时订单统计例子把 Timestamp、Watermark、keyBy、Window、State 串起来。

学习策略：这一讲不要先背 API。先把“业务时间、乱序、等待多久、什么时候结算”的逻辑想清楚，再看 WatermarkStrategy 代码。

2. 为什么流处理里的“时间”会变复杂？

批处理时，我们往往拿到一个已经结束的数据集。即使数据顺序乱了，也可以先排序再计算。流处理不同：输入可能永远不会结束，系统无法等到“所有数据都到齐”以后再开始算。

假设一个订单真实发生在 10:01:00，但用户手机网络不稳定，10:06:00 才被 Kafka/Flink 收到。此时至少有两个时间：

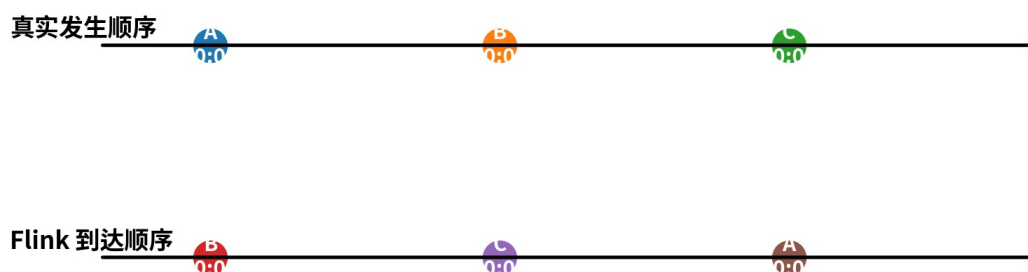
时间	含义	本例
Event Time（事件时间）	业务事件真实发生的时间	10:01:00
Processing Time（处理时间）	Flink 算子实际处理记录时的机器时间	约 10:06:00

如果题目是“统计 10:00-10:05 的销售额”，从业务语义看，这个订单应该属于 10:00-10:05 窗口，因为它在 10:01 真实发生。若按 Processing Time，它会被错误地放到更晚的窗口。

底层原则：Event Time 回答“事情什么时候发生”；Processing Time 回答“计算机什么时候处理”。业务统计通常更关心前者，系统性能监控常更关心后者。

3. 乱序事件是怎么产生的？

乱序：到达顺序 $\neq$ 事件发生顺序



如果只按到达时间统计“10:01~10:02”的订单，A 可能被错分到更晚窗口。

图 2 事件发生顺序与到达顺序可能完全不同

真实系统中乱序不是异常，而是常态。常见原因包括：

- 不同设备网络延迟不同：手机 A 走 5G，手机 B 暂时断网。
- Kafka 多分区并行：不同分区的信息没有全局严格到达顺序。
- 上游重试：某条消息失败后重发，可能晚于后面的消息。
- 批量上传：IoT 设备离线后恢复，一次性补发过去几分钟的数据。
- 分布式调度差异：不同 Source Subtask、网络链路和机器负载不同。

常见误区：“Kafka 是有序的，所以 Flink 不会乱序”是错误的。Kafka 通常只能保证单个 partition 内的局部顺序；跨 partition 本来就没有天然全局顺序，而且业务事件产生时间也可能与写入 Kafka 的时间不同。

4. Processing Time：最简单，但业务语义可能不稳

Processing Time 直接使用算子处理事件时的系统时钟。优点是简单、延迟低，不需要处理乱序时间戳。缺点是结果会受网络、负载、重启、积压等运行状态影响。

特点	Processing Time
时间来源	处理记录时机器的当前时钟
是否需要事件携带时间戳	不需要
是否天然处理乱序	不需要判断事件发生顺序，但业务语义可能偏离真实发生时间
优点	简单、直接、通常低延迟
风险	同一份业务数据在不同负载/延迟下，可能落入不同时间窗口

例如监控“机器当前每 10 秒处理多少条记录”时，Processing Time 就很自然；但统计“用户在 10:00-10:05 真实下了多少单”，通常应使用 Event Time。

5. Event Time：让计算遵循业务世界的时间

Event Time 通常来自事件自身字段，例如：

```
order_id,user_id,amount,event_time
A1001,U01,199.0,2026-09-28 10:01:03
A1002,U02, 89.0,2026-09-28 10:02:17
```

Flink 需要知道哪一个字段代表 Event Time，因此需要 Timestamp Assigner（时间戳提取器）。在传统 DataStream API 中，Timestamp Assigner 通常与 WatermarkGenerator 一起封装在 WatermarkStrategy 里。

关键区别：“给记录赋 Event Time”解决的是“它属于哪个时间位置”；“生成 Watermark”解决的是“系

统认为事件时间整体推进到哪里”。这两个问题不能混成一个。

6. Watermark 到底是什么？

Watermark：对“事件时间已经推进到哪里”的一种声明

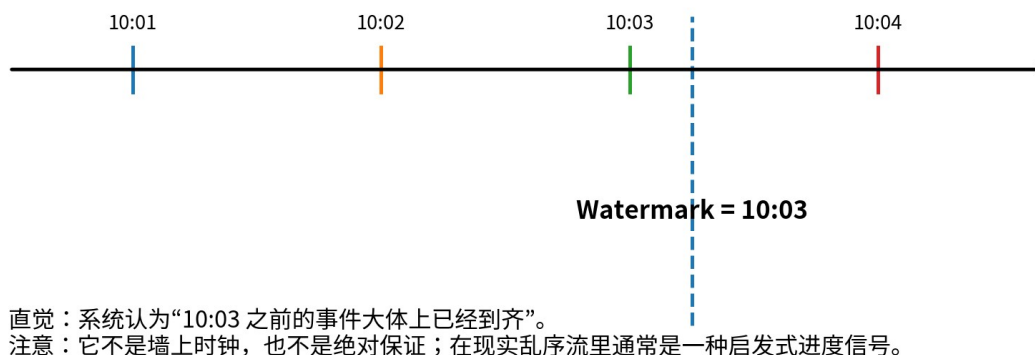


图 3 Watermark 表示事件时间进度，而不是墙上时钟

Flink 官方对经典 Event Time Watermark 的核心语义是：Watermark 向下游表达事件时间已经推进到某个时间点，算子可以据此推进事件时间定时器与窗口。现实流中 Watermark 常是基于乱序上界的启发式估计，而不是“未来绝不再来旧事件”的物理保证。

初学时可以把 Watermark 理解成一句话：

直觉定义：Watermark = 系统对“事件时间已经走到哪里”的进度声明。

例如 Watermark = 10:03，可以粗略理解：系统认为 10:03 之前的事件应该已经基本到齐，因此可以开始安全地推进依赖这些时间点的计算。

但要特别注意：Watermark 不等于“当前机器时间 10:03”，也不等于“10:03 以前的迟到事件以后绝对不会出现”。

7. 有界乱序 Watermark：最常见的入门模型

假设业务经验告诉我们：绝大多数订单最多乱序 5 秒。一个常见思路是维护“目前观察到的最大 Event Time”，再减去允许的乱序上界。直觉公式可以写成：

$$\text{Watermark} \approx \text{maxObservedEventTime} - \text{maxOutOfOrderness}$$

如果当前看到的最大 Event Time 是 10:05:20，允许最大乱序 5 秒，则 Watermark 大约推进到 10:05:15。

最大已观察 Event Time	允许乱序	直觉 Watermark
10:05:20	5 秒	约 10:05:15
10:10:00	30 秒	约 10:09:30
10:10:00	5 分钟	约 10:05:00

允许乱序越大，通常越能等到晚到数据，但窗口结果也更晚产生；允许乱序越小，结果越快，但晚到事件风险越高。

本质权衡：Watermark 是“结果及时性”和“事件完整性”之间的工程折中。不存在对所有业务都完美的固定值。

8. WatermarkStrategy 代码先看懂意思，不必死记

下面是经典 DataStream API 中一个有界乱序策略的 Java 结构。代码重点是理解语义：从订单中取 eventTime，并允许最多 5 秒乱序。

```
WatermarkStrategy<Order> strategy = WatermarkStrategy
    .<Order>forBoundedOutOfOrderness(Duration.ofSeconds(5))
    .withTimestampAssigner((order, recordTimestamp) -> order.getEventTime());

DataStream<Order> withWatermark =
    orders.assignTimestampsAndWatermarks(strategy);
```

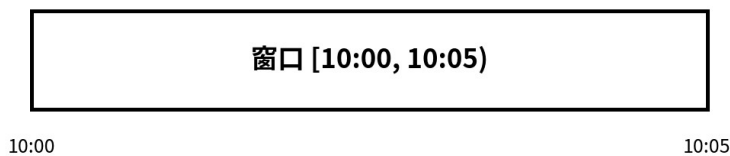
这里的两个角色：

组成	作用
TimestampAssigner	从每条 Order 中提取事件时间戳
WatermarkGenerator	根据观察到的事件时间推进 Watermark
WatermarkStrategy	把二者组合成一套时间策略

版本说明：本课程继续用成熟的经典 DataStream 时间概念建立心智模型。Flink 2.x 的 DataStream API V2 仍处于实验阶段，里面的 Watermark 概念接口有扩展，不适合作为零基础第一入口。

9. Watermark 与 Window 的关系：窗口什么时候触发？

窗口何时“结算”？——看 Watermark 是否越过窗口结束时间



WM=10:04:30 → 还不能认为窗口结束

WM=10:05:00 → 事件时间已跨过窗口边界，可触发窗口计算

迟到事件如何处理，还要看 allowed lateness / side output 等策略；Watermark 不是“迟到事件必然丢弃”的同义词。

图 4 Event Time Window 需要事件时间进度来判断何时结算

假设窗口是 [10:00, 10:05)。在 Event Time 语义下，系统不能因为机器时间到了 10:05 就立即断言“窗口数据齐了”，因为 10:04:58 发生的订单可能 10:05:20 才到。

因此窗口更关心 Watermark 是否已经推进到窗口边界。Watermark 没越过边界时继续等待；推进到/越过边界后，相应 Event Time 定时器和窗口计算就可以触发。

核心关系：Event Time 决定事件“属于哪个窗口”；Watermark 决定系统“何时认为可以推进这个窗口”。

10. 迟到事件：Watermark 以后来的数据怎么办？

假设 [10:00,10:05) 的窗口已经因为 Watermark 推进而触发，随后又来了一条 event_time=10:04:50 的订单。这就是迟到事件（late event）的典型情况。

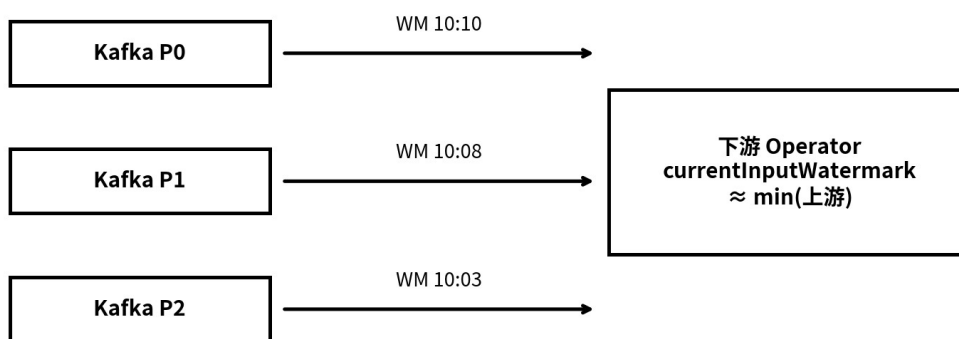
不能简单说“Watermark 以后来的数据全部丢弃”。具体行为取决于窗口配置和 API：可以丢弃，也可以允许一定迟到时间，或者把迟到数据送到 Side Output（侧输出）做补偿处理。

策略	直觉
直接丢弃	结果最快，但可能损失迟到数据
Allowed Lateness	窗口触发后再保留一段时间，允许晚到数据更新结果

重要：Watermark 是进度机制；Late Data Policy 是迟到数据处理策略。二者有关联，但不是同一个概念。

11. 多分区为什么会被最慢的 Watermark 拖住？

多个上游分区时：下游进度通常受“最慢 Watermark”限制



因此 P2 如果长期无数据/卡住，可能拖住整个下游事件时间。
实际系统常需考虑 idleness（空闲分区检测）等机制。

图5 多输入时，下游 Event Time 进度通常由最落后的输入控制

如果一个下游算子接收多个上游 channel，它不能只看最快的那个，因为慢 channel 里还可能出现更早的事件。Flink 的 task 暴露 currentInputWatermark 指标，它体现的是所接收上游 Watermark 中的最低值。

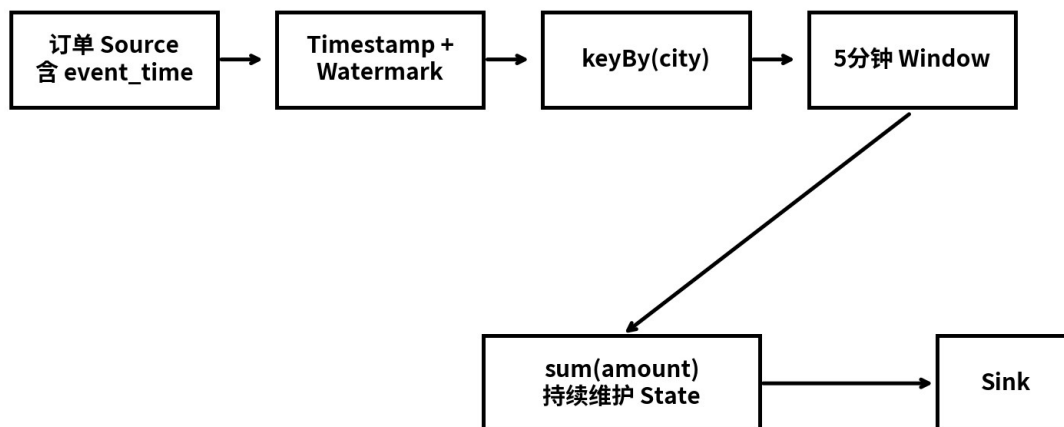
这带来一个现实问题：某个 Kafka 分区暂时没有数据，如果它一直不推进 Watermark，就可能拖住下游窗口。Flink 因此提供 idleness 检测，让长期无事件的输入暂时标记为空闲，避免它永久拖住整体 Event Time。

```
WatermarkStrategy<Order> strategy = WatermarkStrategy.  
    .<Order>forBoundedOutOfOrderness(Duration.ofSeconds(5))  
    .withTimestampAssigner((e, ts) -> e.getEventTime())  
    .withIdleness(Duration.ofMinutes(1));
```

调试入口：以后在 Flink Web UI / Metrics 中看到窗口迟迟不触发，可以检查 currentInputWatermark，判断是不是某个上游分区的 Watermark 卡住。

12. 完整例子：实时统计每个城市每 5 分钟销售额

实时订单统计：从乱序事件到窗口结果



核心关系：Event Time 决定“属于哪个时间窗口”；Watermark 决定“窗口什么时候可以往前推进/触发”；State 保存窗口尚未结算前的中间结果。

图 6 时间语义与窗口、State 的完整关系

业务需求：订单持续到来，按“订单真实发生时间”统计每个城市每 5 分钟销售额。假设允许最多 10 秒乱序。

8. Source 持续接收订单，每条订单含 city、amount、event_time。
9. TimestampAssigner 从 event_time 字段提取 Event Time。
10. WatermarkStrategy 根据观察到的事件时间和 10 秒乱序上界推进 Watermark。
11. keyBy(city) 把同一城市的订单路由到同一 Keyed Subtask。
12. 5 分钟 Event Time Window 决定订单属于哪个时间桶。
13. 窗口内部的 State 持续维护 sum(amount)。
14. 当 Watermark 推进到窗口结束边界，窗口触发并输出销售额。
15. 过晚的数据根据迟到策略丢弃、更新窗口或送入 Side Output。

注意：Flink 真正处理每条记录时仍然发生在某个 Processing Time，但业务归属和窗口推进使用 Event Time + Watermark。

13. 手工推演：一组乱序订单到底如何被计算？

窗口：[10:00,10:05)，允许乱序 10 秒。下面按“到达 Flink 的顺序”列出记录：

到达顺序	订单 Event Time	金额	说明
------	---------------	----	----

1	10:01:00	100	正常到达
2	10:04:50	200	正常到达
3	10:05:08	50	已看到更晚 Event Time
4	10:02:30	80	明显乱序：事件发生很早，但现在才到
5	10:05:20	60	事件时间继续推进

当最大 Event Time 到 10:05:08 时，按“最大事件时间 - 10 秒”直觉，Watermark 大约是 10:04:58，因此 [10:00,10:05) 还不能结算。订单 4 虽然晚到，但仍可能被纳入窗口。

当最大 Event Time 到 10:05:20 时，Watermark 大约推进到 10:05:10，已经越过 10:05 窗口边界，于是窗口可以触发。窗口金额应为 $100 + 200 + 80 = 380$ 。

检查自己： 订单 3 (10:05:08) 为什么不属于第一个窗口？因为窗口归属看 Event Time，不看它第几个到达，也不看处理时间。

14. 什么时候用 Event Time，什么时候用 Processing Time？

场景	更常见选择	原因
电商每 5 分钟真实成交额	Event Time	需要按订单真实发生时刻统计
IoT 设备每分钟温度	Event Time	设备可能断网后补发
广告曝光/点击归因	Event Time	网络延迟不应改变业务时间归属
监控 Flink 当前每秒处理量	Processing Time	关注计算系统此刻处理速度
简单低延迟告警且业务不关心迟到	Processing Time 可能足够	实现简单、响应快

选择标准不是“哪个更高级”，而是业务问题到底问的是“真实发生时间”还是“系统处理时间”。

15. 初学者最容易混淆的 10 个问题

- 1. Watermark 是当前时间吗？** 不是。它是事件时间进度信号。
- 2. Event Time 是 Flink 给事件生成的吗？** 通常不是。它来自业务事件本身的时间字段，Flink 负责提取和使用。
- 3. Watermark=10:05 后，10:04 的数据绝对不会来吗？** 现实中仍可能来，因此还需要迟到数据策略。

4. **Watermark 越大越好吗？** 不是。推进过快可能增加迟到数据；太慢又会增加结果延迟和状态保留时间。
5. **Processing Time 不准确吗？** 它对“系统什么时候处理”是准确的，只是不一定符合业务发生时间语义。
6. **乱序只发生在 Kafka 多分区吗？** 不是。网络、重试、设备离线、批量补发等都会导致乱序。
7. **Window 结束时间到了就一定立即输出吗？** Event Time Window 更看 Watermark 是否推进到触发点。
8. **多分区 Watermark 为什么看最慢的？** 因为任何慢输入都可能继续带来较早 Event Time 的事件。
9. **没有事件的分区会怎样？** 可能拖住 Watermark，所以需要 idleness 等机制。
10. **Watermark 是 Flink 独有概念吗？** 不是。事件时间进度/水位线思想在现代流处理系统中非常常见，只是具体 API 不同。

16. 零基础实验：先用“纸上程序”理解 Watermark

本讲先不要求你安装新的 Flink API。用下面这段 Python 模拟“最大 Event Time - 允许乱序”的直觉即可。它不是 Flink 内部源码，但能帮助你建立正确心智模型。

```
events = [
    ("A", 61),    # 10:01:01 -> 为了简化，用从 10:00 开始的秒数
    ("B", 290),   # 10:04:50
    ("C", 308),   # 10:05:08
    ("D", 150),   # 10:02:30, 乱序晚到
    ("E", 320),   # 10:05:20
]

max_lateness = 10
max_event_time = -1

for name, event_time in events:
    max_event_time = max(max_event_time, event_time)
    watermark = max_event_time - max_lateness
    print(name, "event_time=", event_time, "watermark=", watermark)
```

你应该观察到：D 的 event_time 明明只有 150 秒，但它到达时 Watermark 已经由更晚事件推进；这正是“乱序”和“事件时间进度”两个概念的交汇处。

17. 练习题

16. 一个订单 10:01 发生，10:06 才被 Flink 处理。它的 Event Time 和 Processing Time 分别是什么？
17. 业务要求统计 10:00-10:05 的真实订单金额，应优先使用哪种时间语义？为什么？
18. 最大已观察 Event Time 为 10:10:30，允许最大乱序 20 秒。按入门直觉，Watermark 大约是多少？
19. 窗口 [10:00,10:05)，当前 Watermark=10:04:59。窗口应该已经触发吗？为什么？
20. 三个上游分区 Watermark 分别为 10:20、10:18、10:07。下游事件时间大致受谁限制？
21. Watermark 已越过窗口结束时间后又来了一条旧事件，为什么不能简单断言“一定丢弃”？

18. 练习答案

- 22. 1. Event Time $\approx$ 10:01; Processing Time $\approx$ 10:06。
- 23. 2. Event Time, 因为窗口归属应该按订单真实发生时刻, 而不是网络延迟后的处理时刻。
- 24. 3. 约 10:10:10。
- 25. 4. 通常还没有越过 10:05 的窗口结束边界, 因此不能仅凭这个 Watermark 认为窗口完成。
- 26. 5. 受最慢的 10:07 分区限制, 因为它仍可能带来较早事件。
- 27. 6. 因为迟到数据的实际处理还取决于 allowed lateness、side output 等窗口/业务策略。

19. 本讲最后压缩成 8 句话

- 28. 流数据可能永远不结束, 因此不能等“全部数据到齐”再计算。
- 29. Event Time 是业务事件发生时间; Processing Time 是系统处理时间。
- 30. 分布式流里乱序是常态, 不是例外。
- 31. Timestamp Assigner 告诉 Flink 每条事件的 Event Time。
- 32. Watermark 告诉系统事件时间整体推进到了哪里。
- 33. Event Time 决定数据属于哪个窗口, Watermark 帮助决定窗口何时推进/触发。
- 34. Watermark 是及时性与完整性的折中; 迟到数据还需要单独策略。
- 35. 多输入时最慢 Watermark 会限制下游进度, 因此 idleness 很重要。

下一讲: 第 4 讲将专门讲 Window: Tumbling Window、Sliding Window、Session Window, 以及 Window Assigner、Trigger、Evictor 和 State 为什么共同决定“流里的时间桶怎样计算”。

20. 官方资料核对

本讲概念依据 Apache Flink stable documentation: Generating Watermarks、Built-in Watermark Generators、Debugging Windows & Event Time、Execution Mode (Batch/Streaming)。经典 DataStream WatermarkStrategy 仍是理解 Event Time 的成熟入口; DataStream API V2 当前仍标记为 experimental。

参考网址: nightlies.apache.org/flink/flink-docs-stable/docs/dev/datastream/event-time/generating_watermarks/

; nightlies.apache.org/flink/flink-docs-stable/docs/ops/debugging/debugging_event_time/